Doc. No.: WG21/P0270R2
Date: 2016-11-27
Author: Hans-J. Boehm
Email: hboehm@google.com
Audience: CWG, LWG

P0270R2: Removing C dependencies from signal handler wording

This topic was briefly addressed in P0063R0, but the authors subsequently decided to address it separately. This version reflects P0270R0 discussion in Jacksonville, draft revisions in Oulu, as well as P0270R1 and draft revision discussions by SG1 and CWG in Issaquah.

The current C++ working draft restricts signal handlers to functions in the intersection of C and C++. We believe this is dubious for at least three reasons:

- The semantics implicitly change if we refer to a newer version of the C standard. P0063R1 proposes to normatively refer to C11 instead of C99. This, for example, raises the question of weather thread_local variables, which are supported in C11 with slightly different keyword conventions, are now allowed in signal handlers. This was never our intent, since it would preclude some implementation techniques.
- The current wording is not as clear as it should be. For example, Richard Smith points out that C functions, and hence POFs cannot have C linkage, since there is no way to specify C linkage in C.
- The current restrictions don't appear to reflect those that are really necessary or desirable in signal handlers. There is no good reason that signal handlers can't mention `nullptr` for example. The real restrictions we need are that such functions should have C linkage, should not introduce data races (already addressed by the "potentially concurrent" notion in 1.10), and if they avoid certain facilities that are frequently implemented in a signal-unsafe manner, particularly anything that may require a hidden lock.

We propose to remedy this by replacing the notion of a "POF", "plain old function" with a "signal-safe evaluation" defined entirely in C++ terms. The definition we use largely excludes use of the library, thus avoiding any need to reimplement that in a signal-safe manner. This makes the definition rather restrictive. But we believe it remains at least theoretically possible for the user to write very sophisticated signal handlers through the use of atomics.

We expect that, although these changes are clearly substantive, there is no real practical impact. Existing implementations and user code should continue to work unchanged (or remain as broken as they always were.)

We do not exclude lambda expressions from signal handlers, on the assumption that the lambda expression itself should not require memory allocation. We do not say anything about non-function-local statics, since it should not be possible to trigger construction or destruction from within a signal handler.

As far as I can tell, the `mem...` and `str...` functions are not guaranteed to be async-signal-safe in Posix (or C), so it seems presumptive of us to declare them signal-safe here. I thus decided against it.

We somewhat separably propose to make the behavior of non-signal-safe functions in signal handlers undefined, instead of implementation-defined. We are not aware of any implementations that actually attempt to define this.

The restriction on static initialization warrants discussion. We currently allow even the call of a POF that doesn't access any globals to trigger static initialization (3.6.2p4 [basic.start.init]). It's not clear how to make that work if a thread already part way through the initialization receives the signal. This appears to me to be a preexisting problem that this exposes, rather than a problem introduced by this change. It is partially addressed by P0250, which makes this implementation-defined.

It may make sense to eventually relax these restrictions invoked through an explicit synchronous `raise()` call. It's unclear that this is important enough to bother.

These changes are arguably not essential for the adoption of the P0063. But we believe they are highly desirable and should follow P0063 in fairly short order.

Reaction to CWG discussion in Issaquah

The original discussion is reflected in the CWG notes. Specific comments:

- I believe that currently initialization of a non-local variable can be triggered anywhere. I also believe this needs to be changed, since it is esssentially impossible to write conforming code using this kind of initialization. P0250 at least it makes it implementation-defined. I don't think changes proposed here make things worse. I did not change the wording in this area.
- I'm not sure whether the second bullet in the second list should say "odr use" instead of "access". I'm happy to follow CWG's lead, but I'm not sure what the conclusion was. Left as-is for now.
- I agree that the signal function should be declared extern "C". I added that. Please double-check. I suspect this makes

signal() the only such C library function without a corresponding C++-linkage version. I don't see a good reason for that, but we can revisit later. Preseerving the status quo for now.
- I agree that this is really a property of evaluations occurring as part of the signal handler. I attempted to reflect that in the new wording. Please check.

Proposed wording changes:

Thanks to help from the editors, these are roughly relative to the C++17 CD. Note that the underlying text changed substantially in Oulu, in ways that are orthogonal to our goals here.

Update 18.10.5 [csignal.syn] as follows.

In the synopsis, modify the signal declaration as follows:

```
extern "C" void (*signal(int sig, void (*func)(int)))(int);
```

Insert a new paragraph before paragraph 3:

A *plain lock-free atomic operation* is an invocation of a function `f` from Clause 29, such that:

- `f` is the function `atomic_is_lock_free()`, or
- `f` is the member function `is_lock_free()`, or
- for every argument `A` passed to `f` that points to an atomic object, and, if `f` is a non-static member function, for every address `A` of an atomic object on which `f` is invoked, `atomic_is_lock_free(A)` yields true.

Modify the former paragraph 3 as follows.

~~The common subset of the C and C ++ languages consists of all declarations, definitions, and expressions that may appear in a well formed C++ program and also in a conforming C program. A POF ("plain old function") is a function that uses only features from this common subset, and that does not directly or indirectly use any function that is not a POF.~~ An evaluation is *signal-safe* if it includes none of the following:

- a call to any standard library function, except for (a) functions explicitly identified as signal-safe, and (b) ~~except that it may use~~ plain lock-free atomic operations~~.~~ [*Note*: This implicitly excludes the use of `new` and `delete` expressions that rely on a library-provided memory allocator. – *end note*]; ~~A *plain lock-free atomic operation* is an invocation of a function *f* from Clause 29, such that *f* is not a member function, and either *f* is the function `atomic_is_lock_free()`, or for every atomic argument `A` passed to *f*, `atomic_is_lock_free(A)` yields true. All signal handlers shall have C linkage.~~
- an access to an object with thread storage duration;
- the throwing of an exception; or
- the initialization a variable with static storage duration requiring dynamic initialization (3.6.3, 6.7).

~~The behavior of any function other than a POF used as a signal handler in a C++ program is implementation-defined.~~ A signal handler invocation has undefined behavior if it includes an evaluation that is not signal-safe.

Append a new paragraph.

The function `signal()` is signal-safe.

The following functions need to be callable from handlers. We expect that a few other functions should also eventually be declared signal-safe, but these are among the more blatant cases. `abort()` and `quick_exit()` are treated as safe by C.

Add a new paragraph at the beginning of section 18.3.2.4 [numeric.limits.members]:

Each member function defined in this Clause is signal-safe (18.10.5 [csignal.syn]).

Add a sentence at the end of the remarks for _Exit() in 18.5p3 [support.start.term]:

The function `_Exit()` is signal-safe (18.10.5 [csignal.syn]).

Add a sentence at the end of the remarks for abort() in 18.5p5 [support.start.term]:

The function `abort()` is signal-safe (18.10.5 [csignal.syn]).

Add at new element to the end of 18.5p13 (quick_exit()) [support.start.term]. Note that it is not clear that this is actually correct for current implementations that may hold a lock while running handlers. However this requirement is already present in the C11 standard, and it doesn't seem to make much sense to weaken their specifications for their functions. An LWG issue on this has been requested.

Remarks: The function `quick_exit()` is signal-safe (18.10.5 [csignal.syn]) when the functions registered with `at_quick_exit()` are.

Modify 18.9p1 [support.initlist] as follows:

> The header `<initializer_list>` defines a class template and several support functions related to list-initialization (see 8.5.4). All functions specified in this Clause are signal-safe (18.10.5 [csignal.syn]).

Add a new paragraph between paragraphs 9 and 10 of section 20.2.1 [operators]:

> All four of the above comparison operators are signal-safe (18.10.5 [csignal.syn]) when the expression in the *Returns:* element would be signal-safe.

Modify 20.2.4p1 [forward] as follows:

> The library provides templated helper functions to simplify applying move semantics to an lvalue and to simplify the implementation of forwarding functions. All functions specified in this Clause are signal-safe (18.10.5 [csignal.syn]).

20.14 [meta] provides a few actual functions that return constant values. These are also clearly signal-safe. Add a new paragraph at the end of section 20.14 [meta], before 20.14.1:

> All functions specified in this Clause are signal-safe (18.10.5 [csignal.syn]).

Acknowledgment

Clark Nelson provided many useful suggestions, but not necessarily agreement. This version reflects contributions by many other members of CWG and LWG, including Thomas Koeppe and JF Bastien, who helped shepherd it through CWG and LWG and provided additional feedback.